

Ejercitación de complejidad

Parra Sofía

Ejercicio 1:

Demuestre que $6n^3 \neq O(n^2)$.

Supongamos que $6n^3 = O(n^2)$, entonces por definición:

$$\exists c, m > 0 \mid \forall n > m \quad 6n^3 < cn^2$$

Tomando esta desigualdad, considerando que $n^2 > 0$ y dividiendo ambos lados por n^2 :

$$6n^3/n^2 < cn^2/n^2$$

$$6n < c$$

$$n < c/6$$

$c/6$ es una constante determinada, por lo tanto, sin importar que tan grande sea el valor de c que seleccionemos, siempre vamos a poder encontrar un entero n tal que $n \geq c/6$, lo cual es una contradicción.

Por lo tanto, $6n^3 \neq O(n^2)$

Ejercicio 2:

¿Cómo sería un array de números (mínimo 10 elementos) para el mejor caso de la estrategia de ordenación Quicksort(n)?

El mejor caso de quicksort se da cuando el pivote elegido permite separar el array de manera balanceada. Es decir, por cada llamada al algoritmo reducimos el n de entrada a aproximadamente $n/2$.

En la implementación del libro, se elige como pivote al último elemento del array, por lo tanto, el valor del último elemento de este (y de los subarrays producidos con las llamadas recursivas) debe estar en el medio de los demás elementos:

1	2	5	4	3	7	8	11	10	9	6
---	---	---	---	---	---	---	----	----	---	---

Ejercicio 3:

Cuál es el tiempo de ejecución de la estrategia **Quicksort(A)**, **Insertion-Sort(A)** y **Merge-Sort(A)** cuando todos los elementos del array A tienen el mismo valor?

n: tamaño del array A

Quicksort(A)	$T(n)=\Theta(n^2)$ Porque $T(n)=T(n-1)+f(n)$, es decir, los subarrays quedan totalmente desbalanceados y cae en uno de los peores casos para este algoritmo
Insertion-Sort(A)	$T(n)=\Theta(n)$ Porque de acuerdo con la implementación del libro, el segundo bucle (anidado) no se ejecutaría ya que $A[i] >$ elemento nunca se cumpliría. Estaría dentro del mejor caso
Merge-Sort(A)	$T(n)=\Theta(n \log n)$ Porque el array se divide igual y se recorren todos los elementos al hacer merge

Ejercicio 4:

Implementar un algoritmo que ordene una lista de elementos donde siempre el elemento del medio de la lista contiene antes que él en la lista la mitad de los elementos menores que él. Explique la estrategia de ordenación utilizada.

Ejemplo de lista de salida

7	3	2	8	5	4	1	6	10	9
---	---	---	---	---	---	---	---	----	---

MitadMenores

Primero se toma el primer elemento $A[0]$ como pivote y los demás elementos se separan según su relación con el pivote en dos listas distintas. Si un elemento es menor que el pivote, se agrega a menores y si es mayor o igual, se agrega a resto.

Una vez separados, finalmente se concatena todo. Se toma la mitad de los elementos menores y se coloca antes del pivote, mientras que la segunda parte se coloca después del pivote.

De la misma forma, se distribuye resto entre antes y después del pivote.

Para cualquier caso (mejor, promedio o peor) la complejidad queda definida por el recorrido de los elementos de la lista, así que en todos ellos la complejidad es lineal

Código comentado:

```
def MitadMenores(A:list) -> list:

    # Si hay menos de 3 elementos, devuelve la lista sin modificar

    if len(A)<3:

        return A

    menores=[]

    resto=[]

    # El primer elemento de la lista se toma como pivote, luego se
separan los elementos según sean menores o no que el primero

    for i in A[1:]:

        if i<A[0]:

            menores.append(i)

        else:

            resto.append(i)

    # Coloca la mitad de cada grupo antes y la otra mitad después del
primer elemento. Se usa round para la lista resto y se trunca la lista
menores debido a que si ambas tuvieran un número impar de elementos y
usáramos la misma operación para las dos, el pivote no quedaría al
medio.

    return menores[:len(menores)//2] + resto[:round(len(resto)/2)] +
[A[0]] + menores[len(menores)//2:] + resto[round(len(resto)/2):]
```

Ejercicio 5:

Implementar un algoritmo **Contiene-Suma(A,n)** que recibe una lista de enteros A y un entero n y devuelve True si existen en A un par de elementos que sumados den n. Analice el costo computacional.

ContieneSuma:

En todos los casos la complejidad está dada por un factor de $n \log n$, determinado por el ordenamiento de la lista antes de trabajar con ella. Las comparaciones en sí están definidas por $\theta(n)$, pero la complejidad está dada, como se dijo, por el ordenamiento.

Código comentado:

```
def ContieneSuma(A:list, n:int)->bool:

    # Si hay menos de dos elementos, no puede existir un par

    if len(A)<2:

        return False

    # Ordena la lista para poder utilizar dos punteros (la función sort
    # de python ordena con complejidad  $n \log n$ )

    A.sort()

    menor=0

    mayor=len(A)-1

    # Compara el menor y el mayor elemento mientras no se crucen

    while menor!=mayor:

        # Si la suma es menor que n, aumenta el menor

        if A[menor]+A[mayor]<n:

            menor+=1

        # Si la suma es mayor que n, disminuye el mayor

        elif A[menor]+A[mayor]>n:

            mayor-=1
```

```
# Si la suma coincide con n, encontró un par

else:

    return True

# No se encontró ningún par que sume n

return False
```

Ejercicio 6:

Investigar otro algoritmo de ordenamiento como BucketSort, HeapSort o RadixSort, brindando un ejemplo que explique su funcionamiento en un caso promedio. Mencionar su orden y explicar sus casos promedio, mejor y peor.

Radix Sort con Counting Sort

Radix Sort es un algoritmo de ordenamiento que utiliza Counting Sort como subalgoritmo para ordenar los elementos según cada uno de sus dígitos, comenzando por el menos significativo. Counting Sort tiene un orden de $O(n+k)$ para cualquier caso. Como Radix Sort aplica Counting Sort una vez por cada dígito, su orden es $O(d(n+k))$, donde:

n: cantidad de elementos a ordenar.

k: base en la que están representados los números. Para números decimales, $(k=10)$.

d: cantidad de dígitos del número con mayor cantidad de dígitos.

Caso promedio: Una lista de números que tenga una cantidad de dígitos similar y que no esté necesariamente ordenada: [329,457,657, 839, 436, 720, 355]

Radix Sort comienza ordenando los números según las unidades utilizando Counting Sort. Luego, mantiene ese orden y vuelve a aplicar Counting Sort sobre las decenas y finalmente sobre las centenas. Al finalizar todas las pasadas, los números quedan ordenados:

[720,355,436,457,657,329,839] → [720,329,436,839,355,457,657] →

[329, 355, 436, 457, 657, 720, 839]

Mejor y peor caso: tienen el mismo orden que el caso promedio $O(d(n+k))$. Esto se debe a que Radix Sort realiza las mismas pasadas sobre los dígitos independientemente de si la lista original está ordenada o no.

Por lo tanto, a diferencia de otros algoritmos de ordenamiento, en Radix Sort la distribución inicial de los elementos no produce una diferencia significativa en su cantidad de operaciones, sino que el algoritmo debe procesar todos los elementos en cada una de las d pasadas. Cuando d y k son constantes, este comportamiento puede aproximarse a un orden lineal $O(n)$.

Ejercicio 7:

A partir de las siguientes ecuaciones de recurrencia, encontrar la complejidad expresada en $\Theta(n)$ y ordenarlas de forma ascendente respecto a la velocidad de crecimiento. Asumiendo que $T(n)$ es constante para $n \leq 2$. Resolver 3 de ellas con el método maestro completo: $T(n) = a T(n/b) + f(n)$ y otros 3 con el método maestro simplificado: $T(n) = a T(n/b) + n^c$

a. $T(n) = 2T(n/2) + n^4$

$a = 2, b = 2, c = 4$. Calculamos $n^{\log_b a} = n^{\log_2 2} = n^1$

Como $c = 4 > 1$, se aplica el caso simplificado resultando en una complejidad de: $T(n) = \Theta(n^4)$

b. $T(n) = 2T(7n/10) + n$

$a = 2, b = 10/7, f(n) = n$

$$n^{\log_b a} = n^{\log_{10/7} 2} \approx n^{1,943}$$

Para que se cumpla el primer caso, se requiere que $f(n) = O(n^{\log_b a - \epsilon})$ para algún valor de $\epsilon > 0$.

Buscamos que $1 \leq 1,943 - \epsilon$. Si seleccionamos un valor de $\epsilon = 0,9$:

$$f(n) = n = O(n^{1,943 - 0,9}) = O(n^{1,043})$$

Determinamos entonces que se cumple el Caso 1 con $\epsilon = 0,9$.

En consecuencia, la complejidad es: $T(n) = \Theta(n^{\log_{10/7} 2}) \approx \Theta(n^{1,943})$

c. $T(n) = 16T(n/4) + n^2$

$a = 16, b = 4, f(n) = n^2$. Calculamos $n^{\log_b a} = n^{\log_4 16} = n^2$

Como $f(n) = n^2 = \Theta(n^{\log_b a})$, la complejidad es $T(n) = \Theta(n^2 \log n)$

d. $T(n) = 7T(n/3) + n^2$

$a = 7, b = 3$ y $f(n) = n^2$

$$n^{\log_b a} = n^{\log_3 7} \approx n^{1,771}$$

Para que se aplique el tercer caso, se requiere que $f(n) = \Omega(n^{\log_b a + \epsilon})$ para algún $\epsilon > 0$.

Buscamos cumplir la condición: $2 \geq 1,771 + \epsilon$. Al seleccionar un valor de $\epsilon = 0,2$:

$$f(n) = n^2 = \Omega(n^{1,771 + 0,2}) = \Omega(n^{1,971})$$

Evaluamos la condición de regularidad: $a \cdot f(n/b) \leq k \cdot f(n)$, con $k < 1$.

$$7 \cdot (n/3)^2 = (7/9)n^2 \approx 0,778 \cdot n^2 \leq k \cdot n^2, \text{ donde } k = 0,8 < 1.$$

Determinamos que se cumple el Caso 3 con $\epsilon = 0,2$ y $k = 0,8$.

Así, la complejidad es: $T(n) = \Theta(n^2)$

e. $T(n) = 7T(n/2) + n^2$

$a = 7, b = 2, c = 2$. Calculamos $n^{\log_b a} = n^{\log_2 7} \approx n^{2,807}$

Como $c = 2 < 2,807$, resulta una complejidad de $T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2,807})$

f. $T(n) = 2T(n/4) + \sqrt{n}$

$$T(n) = 2T(n/4) + \sqrt{n}$$

$a = 2, b = 4, f(n) = n^{1/2}$

$$n^{\log_b a} = n^{\log_4 2} = n^{1/2}$$

Para la aplicación del segundo caso, se exige que $f(n) = \Theta(n^{\log_b a})$.

$$f(n) = n^{1/2} = \Theta(n^{1/2})$$

Así, la complejidad es: $T(n) = \Theta(\sqrt{n} \log n)$

A tener en cuenta:

1. Usen lápiz y papel primero

2. ~~No se puede utilizar otra Biblioteca mas alla de algo1.py y linkedlist.py~~

3. Hacer una análisis por cada algoritmo implementado del caso mejor, el caso peor y una perspectiva del caso promedio.